

Object-Oriented Programming

Classes · Objects · Members · Constructors · Static · Testing · Patterns · Packages

Faculty of Information Technologies
Industrial University of Ho Chi Minh City

Chapter Contents

01 Object-Oriented Programming

02 Designing a Class

03 Implementing Instance Members

04 Constructors & Object Initialization

05 Static Variables & Methods

06 Testing & Debugging Classes

07 Object References & Design Patterns

08 Packages

01 · OBJECT-ORIENTED PROGRAMMING

Object-Oriented Programming

What it is · Objects · Classes · Relationship

What is Object-Oriented Programming?

A programming paradigm that organizes software design around data (objects) rather than functions and logic.

Programs are built by creating objects that interact with each other.

Encapsulation

Hide internal details
Expose only what's
needed

Abstraction

Focus on essential
features
Ignore irrelevant
details

Inheritance

Reuse & extend
existing classes

Polymorphism

Same interface
different behaviors

Why Object-Oriented Programming?

01 **Modularity**
Code is organized into self-contained objects → easier to understand and maintain

02 **Reusability**
Classes can be reused across projects; inheritance reduces duplication

03 **Scalability**
Large systems grow more naturally by adding new objects and interactions

04 **Maintainability**
Changes in one class have limited impact if encapsulation is respected

Objects and Classes

Class

Blueprint / Template

- Defines the structure
- Attributes (data)
- Methods (behavior)
- Does not occupy memory for instance data until an object is created

Object

Instance of a Class

- Concrete entity in memory
- Has its own state (values)
- Can perform actions
- Many objects can be created from one class

Class → Object Relationship

Class: Car

color, brand, speed
start(), stop(), accelerate()

instantiate (new)

Object: car1

color = red
brand = Toyota
speed = 0

Object: car2

color = blue
brand = Honda
speed = 60

Object: car3

color = black
brand = BMW
speed = 120

Real-world Analogy

Class $\approx$ Blueprint

Architect's plan of a house — defines rooms, doors, materials. No physical house yet.

Object $\approx$ House

An actual built house following the blueprint. Many houses can be built from the same plan.

Attributes $\approx$ Features

Color of walls, number of rooms, address — the current state of that house.

Methods $\approx$ Actions

Open door, turn on lights, lock — things the house (or its owner) can do.

State and Behavior

State

What the object knows

- Data stored inside the object
- Represented by instance variables (attributes / fields)
- Can change over time

Example (Car):

color, brand, speed, fuelLevel

Behavior

What the object can do

- Actions the object can perform
- Represented by methods
- Can read / modify state

Example (Car):

start(), stop(), accelerate()
refuel(), getSpeed()

Basic Structure of a Java Class

```
public class Car {  
    // 1. Fields (State)  
    private String color;  
    private String brand;  
    private int speed;  
  
    // 2. Constructor  
    public Car(String color, String brand) { ... }  
  
    // 3. Methods (Behavior)  
    public void accelerate(int amount) { ... }  
    public int getSpeed() { return speed; }  
}
```

Key Takeaways

- 01 OOP organizes code around objects that combine data and behavior
- 02 Four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism
- 03 A Class is a blueprint; an Object is a concrete instance
- 04 One class can create many independent objects, each with its own state
- 05 State = fields; Behavior = methods
- 06 Next: Designing a Class

02 · DESIGNING A CLASS

Designing a Class

Simple Class · Instance Variables · Implementation

Implementing a Simple Class

01

Identify the entity

What real-world thing are you modeling? (Student, Book, Account...)

02

Decide the state

What data does each object need to remember?

03

Decide the behavior

What actions can the object perform?

04

Write the class

Declare fields, constructor(s), and methods

Designing a Class — From Concept to Code

1. Concept

What real-world
entity?
Student ·
Account · Course



2. Responsibilities

What must it
know?
What must it
do?



3. Public API

Methods clients
may call
deposit ·
getBalance...



4. Data + Code

private fields
constructors ·
methods

Example: A Simple Student Class

```
public class Student {  
    // Instance variables (state)  
    private String name;  
    private String studentId;  
    private double gpa;  
  
    // Simple constructor  
    public Student(String name, String studentId) {  
        this.name = name;  
        this.studentId = studentId;  
        this.gpa = 0.0;  
    }  
}
```

Instance Variables

Instance variables (fields) store the state of each individual object. Every object gets its own copy.

Per-object

Each object has its own values. Changing one does not affect others.

Declared in class

Usually private. Initialized in constructor or with default values.

Lifetime

Exist as long as the object exists (until garbage collected).

Declaring Instance Variables

```
// Good practice
private String name;
private int age;
private boolean active;

// With default value
private double balance = 0.0;

// Avoid public fields
// public String name; ← bad
```

Access modifier

Prefer private → encapsulation

Naming

camelCase, meaningful names

Type

Choose the most suitable type

Initialization

Constructor or field initializer

Specifying the Public Interface

The public interface is the set of public methods (and constants) that clients of the class are allowed to use. It defines “what the class can do” from the outside.

What to expose

Only the operations that make sense for clients.
Keep it small and clear.

What to hide

Internal fields and helper methods. Clients should not depend on them.

Contract

Public methods form a contract: inputs, outputs, and expected behavior.

Public Interface — Example (BankAccount)

Public Interface

```
deposit(amount)
withdraw(amount)
getBalance()
getAccountNumber()
```

- Clients only see these
- Clear and stable API

Hidden Internals

```
private double balance
private String accountNumber
private void logTransaction()
private boolean validateAmount()
```

- Not visible to clients
- Can change freely later

Designing the Data Representation

How will the object store its state internally? Choose fields that support the public interface efficiently and correctly.

01 Support the interface

Every public method must be implementable with the chosen fields.

02 Keep it minimal

Store only what is necessary. Avoid redundant data.

03 Choose good types

Use the most precise type (int vs double, String vs enum...).

04 Think of invariants

What must always be true about the data? Enforce it.

Example: Data Representation Choices

Option A — Simple

```
class Time {  
    private int hours;  
    private int minutes;  
    private int seconds;  
}
```

Easy to read & write
Need care with overflow

Option B — Total seconds

```
class Time {  
    private int totalSeconds;  
}
```

Simpler arithmetic
Convert for display
Fewer invariants

Encapsulation

Encapsulation = bundling data + methods that operate on that data, and restricting direct access to the internal state.

Goal: protect the integrity of the object and hide implementation details.

Hide data

Make fields private.
Outside code cannot touch them directly.

Control access

Provide public getters / setters only when needed, with validation.

Hide complexity

Clients call simple methods; internal logic can be complex.

Encapsulation — Visual Model

Class: BankAccount

Public Interface (visible to clients)

deposit() withdraw() getBalance() getAccountNumber()

Private Implementation (hidden)

balance accountNumber logTransaction()
validateAmount()

Benefits of Encapsulation

01

Data protection

Invalid values cannot be assigned directly. Validation lives in one place.

02

Flexibility

You can change internal representation without breaking client code.

03

Reduced coupling

Clients depend only on the public interface, not on private fields.

04

Easier maintenance

Bugs related to state are localized inside the class.

Key Takeaways

- 01 Design steps: entity → state → behavior → code
- 02 Public interface = methods clients may call
- 03 Choose minimal fields that support the interface
- 04 Encapsulation: private fields + controlled access
- 05 Good encapsulation → flexibility and easier maintenance
- 06 Next: Implementing Instance Members

03 · IMPLEMENTING INSTANCE MEMBERS

Implementing Instance Members

Variables · Methods · `this` · Privacy · Public API

Implementing Instance Members

After designing the public interface and data representation, implement the methods that form that interface: getters, setters, and other behaviors.

Getters

Return the current value of a field (read access).

Setters

Update a field with validation (write access).

Other methods

Business logic that uses or changes the object's state.

Getters (Accessors)

Purpose

- Provide controlled read access
- Usually named getXxx()
- Often simply return the field
- Can compute a derived value

Do not always create a getter for every field — only when clients need the information.

Example

```
public double getBalance() {  
    return balance;  
}  
  
public String getOwner() {  
    return ownerName;  
}  
  
// Derived value  
public boolean isEmpty() {  
    return balance == 0;  
}
```

Setters (Mutators)

Purpose

- Provide controlled write access
- Usually named setXxx()
- Ideal place for validation
- Can reject illegal values

Not every field needs a setter.
Some values are set only at
construction time.

Example with validation

```
public void setBalance(double amount) {  
    if (amount < 0) {  
        throw new IllegalArgumentException(  
            "Balance cannot be negative");  
    }  
    this.balance = amount;  
}
```

Other Instance Methods

Methods that implement real behavior — not just get/set. They use the object's state and often change it.

```
public void deposit(double amount) {  
    if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");  
    balance += amount;  
}
```

```
public boolean withdraw(double amount) {  
    if (amount <= 0 || amount > balance) return false;  
    balance -= amount;  
    return true;  
}
```

Accessing Object State

Inside the class

Methods can access fields directly by name:

```
public void deposit(double amount) {  
    balance = balance + amount;  
}
```

No need for getter here.

Outside the class

Clients must use public methods only:

```
BankAccount acc = new ...;  
acc.deposit(100);  
double b = acc.getBalance();  
  
// acc.balance = 999; ← Error!
```

The this Reference

this is a reference to the current object — the object on which the method was called.

1. Disambiguate names

```
public void setBalance(double balance) {  
    this.balance = balance;  
}
```

this.balance → field
balance → parameter

2. Pass current object

```
public void printInfo() {  
    Printer.print(this);  
}
```

Useful when another method
needs a reference to you.

Method Invocation: `object.method()` → `this`

When you write `obj.deposit(100)`, Java calls `deposit` on that object. Inside the method, `this` refers to `obj`.

Client code

```
acc.deposit(100);
```

JVM resolves

```
find deposit on  
BankAccount
```

Inside method

```
this == acc  
this.balance += 100
```

Keeping Instance Variables Private

Protect integrity

Prevent clients from putting the object into an invalid state.

Hide representation

You can change fields later without breaking client code.

Single point of control

All changes go through methods → easy to add validation or logging.

Clearer API

Clients only see the public methods, not the internal details.

Defining Public Methods

01

Clear names

Method name should describe the action (deposit, withdraw, getBalance).

02

Minimal parameters

Pass only what is needed. Prefer simple types when possible.

03

Validate inputs

Check arguments early and reject illegal values.

04

Document the contract

What does the method require? What does it guarantee?

Putting It Together — BankAccount

```
public class BankAccount {  
    private String accountNumber;  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) this.balance += amount;  
    }  
    public boolean withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount; return true;  
        }  
        return false;  
    }  
    public double getBalance() { return balance; }  
}
```

Programming with Packages

```
// 1. Declare the package
package com.iuh.fit.banking.model;

// 2. Import what you need
import com.iuh.fit.banking.util.ValidationHelper;
import java.util.ArrayList; // or java.util.*;

// 3. Use the classes
public class BankAccount {
    private List<String> history = new ArrayList<>();
    ...
}
```

Best Practices

- 01 Use reverse-domain naming (`com.company.project`)
- 02 Keep package names lowercase and short
- 03 One public top-level class per source file
- 04 Prefer specific imports over wildcard (`import java.util.*`)
- 05 Place package-private helpers next to the classes that use them
- 06 Match directory structure exactly to package name

Key Takeaways

- 01 Keep instance variables private; access via methods
- 02 Getters read state; setters update with validation
- 03 `this` refers to the current object (the receiver of the call)
- 04 Override `toString()` for readable output; `equals()` for meaning
- 05 Public methods form the stable client API
- 06 Next: Constructors & Object Initialization

04 · CONSTRUCTORS & OBJECT INITIALIZATION

Constructors & Object Initialization

What · Declaring · Overloading · Initialization

What is a Constructor?

A constructor is a special method that is called automatically when an object is created with `new`. Its job is to initialize the object's state.

Same name as class

Must match the class name exactly

No return type

Not even void — constructors don't return values

Called by `new`

Invoked automatically when the object is created

Initializes state

Sets initial values for instance variables

Why Do We Need Constructors?

- 01** **Guarantee valid state**
Object is never left with uninitialized or illegal values.
- 02** **Required data at creation**
Force the client to provide essential information (e.g. account number).
- 03** **Encapsulation**
Clients don't need to call many setters right after new.
- 04** **One-time setup**
Perform initialization logic that should run only once.

Declaring Constructors

```
public class BankAccount {  
    private String accountNumber;  
    private double balance;  
  
    // Constructor  
    public BankAccount(String accountNumber, double initialBalance) {  
        this.accountNumber = accountNumber;  
        if (initialBalance < 0) {  
            throw new IllegalArgumentException("Balance cannot be negative");  
        }  
        this.balance = initialBalance;  
    }  
}
```

The Default Constructor

If you write no constructor at all, Java automatically provides a public no-argument constructor that does nothing (fields get default values: 0, null, false...).

When you write none

Java inserts:

```
public ClassName() { }
```

Fields start with default values.

When you write any

The automatic default constructor disappears.

You must provide a no-arg one yourself if needed.

Constructor Overloading

A class can have multiple constructors as long as their parameter lists are different (different number or types of parameters).

```
public BankAccount(String accountNumber) {           // balance = 0
    this(accountNumber, 0.0);
}

public BankAccount(String accountNumber, double balance) {
    this.accountNumber = accountNumber;
    this.balance = balance;
}
```

Calling Another Constructor — `this()`

One constructor can call another constructor of the same class using `this(...)`. This avoids code duplication and must be the first statement.

First statement only

`this(...)` must appear as the very first line of the constructor body.

Avoid duplication

Common initialization logic lives in one constructor; others delegate to it.

No recursive loop

Constructors must not call each other in a cycle.

Object Creation Flow

1

new BankAccount(...)

Memory is allocated for the object

2

Fields get defaults

0, null, false... are assigned first

3

Constructor runs

Your initialization code executes

4

Reference returned

The new object is ready to use

The Role of this in Constructors

1. Distinguish field & parameter

```
public Student(String name) {  
    this.name = name;  
}
```

this.name → instance variable
name → parameter

Without this → wrong assignment
(parameter assigned to itself)

2. Call another constructor

```
public Student(String name) {  
    this(name, 0.0); // call other ctor  
}
```

```
public Student(String name, double gpa) {  
    this.name = name;  
    this.gpa = gpa;  
}
```

this(...) must be the first statement

Constructor vs. Method

Aspect	Constructor	Method
Name	Same as the class name	Any name (usually a verb)
Return type	None (not even void)	Required (or void)
When called	Automatically by new	Explicitly by the programmer
Purpose	Initialize a new object	Perform an action / behavior
Inheritance	Cannot be inherited	Can be overridden

Initializing Objects

1. Field initializer

```
private int count = 0;
```

Runs before the constructor body.

2. Constructor body

```
this.name = name;
```

Most common and flexible way.

3. Initialization block

```
{ ... }
```

Runs every time an object is created, before the constructor.

4. Default values

If nothing is assigned → 0 / null / false depending on the type.

Object References

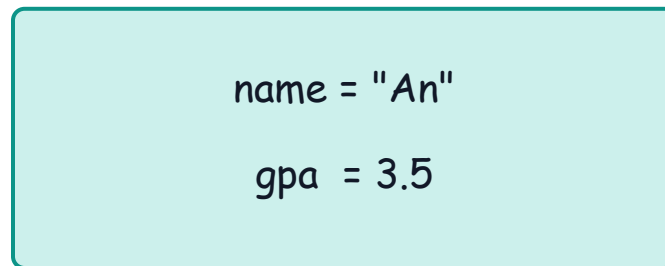
A variable of a class type does not hold the object itself — it holds a reference (address) pointing to the object in the heap. Multiple variables can refer to the same object.

Stack (variables)



both hold the same address

Heap (object)



One Student object in memory

Common Initialization Mistakes

X Forgetting this when names clash

✓ `this.name = name;` (not `name = name;`)

X No validation of input values

✓ Validate inside the constructor; throw if illegal

X Making fields public and assigning outside

✓ Keep fields private + initialize in constructor

X Calling an overridable method in a constructor

✓ Avoid — the subclass is not fully ready yet

X Forgetting to initialize collections / arrays

✓ `new ArrayList<>()` or `new Type[size]` inside the constructor

Key Takeaways

- 01 Constructor initializes a new object — same name, no return type
- 02 Overload constructors; use `this(...)` to avoid duplication
- 03 A class variable holds a reference, not the object itself
- 04 Validate inputs in constructors; keep fields private
- 05 Avoid calling overridable methods from constructors
- 06 Next: Static Variables & Methods

05 · STATIC VARIABLES & METHODS

Static Variables & Methods

Instance vs Static · Class-level data & behavior

Instance Members vs. Static Members

Instance Members

- Belong to each object
- One copy per instance
- Accessed via object reference
`obj.field / obj.method()`
- Different values for different objects

Examples:

`balance, name, getBalance()`

Static Members

- Belong to the class itself
- Only one shared copy
- Accessed via class name
`ClassName.field / ClassName.method()`
- Same value for all objects

Examples:

`interestRate, accountCount, Math.sqrt()`

Static Variables

A static variable (class variable) is shared by all instances of the class. There is only one copy in memory.

```
public class BankAccount {  
    private double balance;           // instance  
    private static double interestRate = 0.05; // static  
    private static int accountCount = 0;    // static  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
        accountCount++; // shared counter  
    }  
}
```

Class-level Data — Visual Model

Class BankAccount (shared)

interestRate = 0.05
accountCount = 3

acc1

balance = 1000

shares interestRate
& accountCount

acc2

balance = 2500

shares interestRate
& accountCount

acc3

balance = 500

shares interestRate
& accountCount

Static Methods

A static method belongs to the class. It can be called without creating an object and can only access static members directly.

```
public static double getInterestRate() {  
    return interestRate;    // OK - static  
}  
  
public static void setInterestRate(double rate) {  
    interestRate = rate;  
}  
  
// Call without an object:  
BankAccount.setInterestRate(0.06);
```

Class-level Data and Behavior

Shared data

Values that are the same for every object (interest rate, configuration, counters).

Utility behavior

Operations that do not depend on a particular object's state (`Math.sqrt`, helpers).

Factory / creation

Methods that create and return objects (often static).

Constants

public static final values that never change (`PI`, `MAX_SIZE...`).

When to Use static

Good candidates

- Constants (static final)
- Shared counters / ID generators
- Utility / helper methods
- Shared configuration values
- Factory methods
- Math-like operations

Avoid when...

- Data differs per object
- Method needs object state
- You want polymorphism
(static cannot be overridden)
- Making everything static
"just for convenience"

Key Takeaways

- 01 Instance members belong to each object; static members to the class
- 02 Static variables: one shared copy (counters, rates, config)
- 03 Static methods cannot use instance state directly
- 04 Use static for constants, utilities, factories — not for per-object data
- 05 static methods are not polymorphic (cannot override)
- 06 Next: Testing & Debugging Classes

06 · TESTING & DEBUGGING CLASSES

Testing & Debugging Classes

Test objects · Methods · Tracing · State · Common bugs

Testing a Class

Testing means systematically checking that a class behaves correctly under different conditions before it is used in a larger program.

Correctness

Methods produce the expected results

Edge cases

Behavior at boundaries (0, null, empty...)

Error handling

Invalid inputs are rejected properly

State integrity

Object stays in a valid state after operations

Creating Test Objects

Create objects with known, controlled initial states so you can predict the expected results of method calls.

```
// Create objects with known state
BankAccount acc1 = new BankAccount("A001", 1000.0);
BankAccount acc2 = new BankAccount("A002", 0.0);    // edge case
BankAccount acc3 = new BankAccount("A003", 50.0);

// Now you know exactly what getBalance() should return
// and how deposit / withdraw should change the state.
```

Testing Instance Methods

01**Arrange**

Create the object with a known initial state

02**Act**

Call the method you want to test

03**Assert**

Check the return value and/or the new state of the object

04**Repeat**

Try normal cases, edge cases, and invalid inputs

Example: Testing deposit & withdraw

```
BankAccount acc = new BankAccount("A001", 100.0);

// Test deposit
acc.deposit(50.0);
System.out.println(acc.getBalance()); // expect 150.0

// Test successful withdraw
boolean ok = acc.withdraw(30.0);
System.out.println(ok);                // expect true
System.out.println(acc.getBalance()); // expect 120.0

// Test failed withdraw (insufficient funds)
ok = acc.withdraw(500.0);
System.out.println(ok);                // expect false
```

Tracing Objects

Tracing means following an object's state step by step as methods are called, so you can see exactly how the data changes.

Create

balance = 100

deposit(40)

balance = 140

withdraw(20)

balance = 120

withdraw(200)

balance = 120 (rejected)

Understanding Object State Changes

Before the call

Know the exact values of all relevant fields.

During the call

Which fields does the method read or write?

After the call

What is the new state? Does it match the specification?

Side effects

Did any other object or static variable change?

Common Object-Related Bugs

NullPointerException

Forgot to create the object or left a reference null

Wrong initial state

Constructor does not set fields correctly

State not updated

Method calculates a value but never assigns it back to a field

Shared mutable state

Static or public fields modified unexpectedly by another part of the program

Aliasing surprises

Two variables point to the same object; changing one affects the other

Key Takeaways

- 01 Test with known initial states and clear expected results
- 02 Follow Arrange → Act → Assert for each method
- 03 Trace object state before, during, and after calls
- 04 Cover edge cases and invalid inputs, not only the happy path
- 05 Watch for null references, missing updates, and aliasing bugs
- 06 Next: Object References & Object Modeling

07 · OBJECT REFERENCES & OBJECT MODELING

Object References & Object Modeling

References · Aliasing · Initialization · Modeling real-world objects

Understanding Object References

A reference is a value that points to an object in memory (the heap). Variables of class types store references, not the objects themselves.

Reference $\neq$ Object

The variable holds an address, not the data.

null means no object

A reference can be null until assigned.

Assignment copies the reference

Both variables then point to the same object.

Reference Variables

```
Student s;           // declaration - currently null
s = new Student("An"); // now points to an object

Student t = s;       // t points to the SAME object

s.setName("Binh");
System.out.println(t.getName()); // "Binh" (shared object)

s = null;           // s no longer points to anything
// t still points to the object
```

Multiple References to the Same Object

When two or more variables refer to the same object, changing the object through one variable is visible through the others. This is called aliasing.

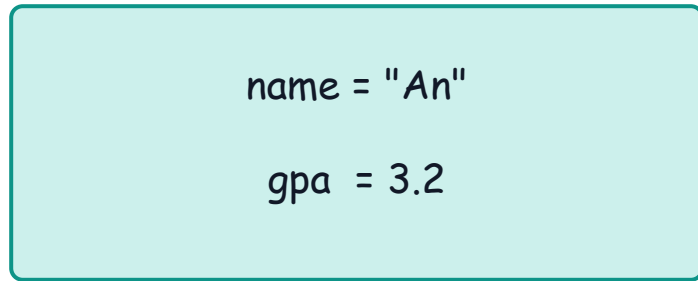
Stack



s1 and s2 are aliases

s2.gpa = 3.2 also changes s1.gpa

Heap



One shared object

Forgetting to Initialize Object References

Using a reference that is still null causes a `NullPointerException` at runtime — one of the most common bugs in Java.

X Problem

```
Student s;  
s.setName("An");  
// NullPointerException  
  
// or field left null  
private Address addr;  
addr.getCity(); // boom
```

✓ Solution

```
Student s = new Student();  
s.setName("An");  
  
// or initialize in constructor  
public Person() {  
    this.addr = new Address();  
}
```

Patterns for Object Data

Simple attributes

Primitive or String fields that describe the object (name, age, balance).

Object composition

A field that is itself an object (Person has an Address).

Collections of objects

Lists or arrays of related objects (Course has a list of Students).

Shared / static data

Class-level values used by all instances (interest rate, counters).

Modeling Real-World Objects

Good object design starts by identifying the real-world entities, their data, and their responsibilities.

1

Identify entities

Student, Course,
BankAccount...

2

List attributes

What data does each
entity need?

3

List behaviors

What can the entity
do?

4

Define relationships

Has-a, uses-a, belongs-
to...

Example: Modeling a Course

Course

Attributes:

- code, title, credits
- List<Student> students

Behaviors:

- enroll(Student)
- drop(Student)
- getEnrollmentCount()

Student

Attributes:

- id, name, gpa

Behaviors:

- getName()
- setGpa(double)

Relationship:

Course has many Students
(composition / aggregation)

Composition vs Aggregation

Composition (strong)

Whole owns the part.
Part's lifetime depends on whole.

Example:
House has Rooms
Order has LineItems

If House is deleted → Rooms go too.

Aggregation (weak)

Whole uses the part.
Part can exist independently.

Example:
Course has Students
Department has Teachers

Student still exists without Course.

Key Takeaways

- 01 References store addresses; assignment shares the same object
- 02 Aliasing: change via one variable is visible through the others
- 03 Uninitialized references → `NullPointerException`
- 04 Composition = strong ownership; Aggregation = weak association
- 05 Model entities by attributes, behaviors, and relationships
- 06 Next: Packages

08 · PACKAGES

Packages

Organization · Access control · import · Best practices

What is a Package?

A package is a namespace that organizes related classes and interfaces. It helps avoid name conflicts and controls access to the code.

Organization

Group related classes together

Name uniqueness

Avoid clashes
(`java.util.List` vs
`myapp.List`)

Access control

package-private
visibility

Scalability

Large projects stay
manageable

Package Specification

```
// First non-comment line of the source file
package com.iuh.fit.banking;

public class BankAccount {
    ...
}

// Directory structure must match:
// src/com/iuh/fit/banking/BankAccount.java

// Convention: reverse domain name
// com.company.project.module
```

Package Access (package-private)

If you omit a public / private / protected modifier, the member has package access — visible only to classes in the same package.

Same package

```
class Helper {           // package-private
    void doWork() { ... } // visible here
}
```

```
// Other class in same package
// can use Helper freely
```

Different package

Helper is NOT visible.
You must make the class public and import it, or place it in the same package.

Organizing Classes into Packages

`com.iuh.fit.banking`

`model`

`BankAccount.java`

`Customer.java`

`service`

`AccountService.java`

`TransferService.java`

`util`

`ValidationHelper.java`

`Main.java` at root · Group by responsibility / layer, not by type alone.

Access Control Summary

Modifier	Same package	Other package	Scope
public	Yes	Yes	Everywhere
protected	Yes	Subclass only	Package + subclasses
(package)	Yes	No	Same package only
private	No*	No	Only inside the class

Programming with Packages

```
// 1. Declare the package
package com.iuh.fit.banking.model;

// 2. Import what you need
import com.iuh.fit.banking.util.ValidationHelper;
import java.util.ArrayList; // or java.util.*;

// 3. Use the classes
public class BankAccount {
    private List<String> history = new ArrayList<>();
    ...
}
```

Best Practices

- 01 Use reverse-domain naming (`com.company.project`)
- 02 Keep package names lowercase and short
- 03 One public top-level class per source file
- 04 Prefer specific imports over wildcard (`import java.util.*`)
- 05 Place package-private helpers next to the classes that use them
- 06 Match directory structure exactly to package name

Key Takeaways

- 01 Packages organize classes and prevent name conflicts
- 02 package declaration must be the first statement in the file
- 03 Directory structure must mirror the package name
- 04 No modifier = package-private (same package only)
- 05 Use import for public classes from other packages
- 06 Chapter complete — review & practice!